

# DSL-Xpert 2.0: Enhancing LLM-driven code generation for domain-specific languages<sup>☆,☆☆</sup>

Victor Lamas<sup>a</sup>, Daniel Garcia-Gonzalez<sup>a</sup>, Luca Sala<sup>b</sup>, Miguel R. Luaces<sup>a</sup>

<sup>a</sup> Universidade da Coruña, Centro de Investigación CITIC, Database Lab., A Coruña, Spain

<sup>b</sup> mim.fscire, Bologna, Italy

## ARTICLE INFO

Dataset link: <https://github.com/lbdudc/dsl-xpert>, <https://github.com/lucasala1997/text-to-DL>

### Keywords:

Domain-specific languages (DSLs)  
Large language models (LLMs)  
Semantic parsing  
Grammar prompting  
Few-shot learning

## ABSTRACT

**Context:** Domain-specific languages (DSLs) are essential for modeling specialized concepts, offering greater fluency and efficiency than general-purpose languages. However, their adoption is often hindered by steep learning curves, limited tools, and complex implementations. While large language models (LLMs) can generate DSL code from natural language, their performance is limited in niche areas due to a lack of training on specific DSL definitions.

**Objectives:** This paper introduces DSL-Xpert 2.0, a tool that addresses these challenges by using LLMs to generate DSL code effortlessly.

**Methods:** Integrating grammar prompting and few-shot learning ensures the effective handling of proprietary DSLs. In addition, advanced features such as automatic grammar validation, input/output correction, and integration with platforms like OpenAI, HuggingFace, and WebLLM provide robust, reliable results while simplifying workflows for novices and experts. To further demonstrate the tool's practical value, this paper provides a running example illustrating its workflow and a complementary user survey conducted across multiple DSLs of varying complexity, following the Technology Acceptance Model (TAM), to evaluate its impact on easing the DSL learning curve.

**Results:** With a user-friendly and flexible design, DSL-Xpert 2.0 supports a wide range of DSL designs with minimal configuration. Its intuitive interface allows developers to focus on innovative problem-solving rather than technical complexities. Findings from the user survey confirm that DSL-Xpert 2.0 effectively reduces the learning effort required to work with DSLs and is perceived as both useful and easy to use. Additionally, this paper provides a detailed performance analysis across various LLMs, showcasing the adaptability and effectiveness of the tool.

**Conclusion:** By simplifying DSL development and lowering entry barriers, DSL-Xpert 2.0 accelerates adoption and innovation, positioning itself as a valuable resource for domain-specific projects.

## 1. Motivation

Domain-specific languages (DSLs) have become indispensable tools for representing and managing specialized concepts within a particular domain. By providing tailored abstractions and constructs, DSLs enable experts to articulate complex processes and requirements with greater clarity and precision [1,2]. Compared to general-purpose programming languages, DSLs simplify workflows by focusing on domain-specific expressiveness, which enhances the reusability, maintainability, and

scalability of software projects [3–5]. Despite these advantages, adopting DSLs is often accompanied by significant challenges. Designing and implementing a DSL requires considerable effort, in addition to technical and specialized knowledge [6,7]. Furthermore, limited support tools, poor documentation, and steep learning curves can deter potential users, hindering adoption in broader contexts [8]. These limitations make DSLs a double-edged sword, offering immense potential but coming at a high cost.

<sup>☆</sup> This article is part of a Special issue entitled: 'Next\_MBSE' published in Information and Software Technology.

<sup>☆☆</sup> This paper is an extended version of our paper published in ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24), September 22–27, 2024, Linz, Austria. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3652620.3687782>.

\* Corresponding author.

E-mail addresses: [victor.lamas@udc.es](mailto:victor.lamas@udc.es) (V. Lamas), [d.garcia2@udc.es](mailto:d.garcia2@udc.es) (D. Garcia-Gonzalez), [luca.sala@unimore.it](mailto:luca.sala@unimore.it) (L. Sala), [miguel.luaces@udc.es](mailto:miguel.luaces@udc.es) (M.R. Luaces).

<https://doi.org/10.1016/j.infsof.2025.107954>

Received 13 March 2025; Received in revised form 27 October 2025; Accepted 28 October 2025

Available online 31 October 2025

0950-5849/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Nowadays, rapid advances in large language models (LLMs), such as GPT-4 [9] or Llama 3 [10], have introduced new possibilities for developing and using DSLs. These models, trained on massive datasets spanning diverse natural and programming languages, possess an exceptional ability to process linguistic and semantic patterns. By taking advantage of this capability, LLMs can help generate DSL code, validate grammar rules, and even create documentation, ultimately reducing development time and complexity [11,12]. However, effective use of LLMs for DSLs is far from straightforward. Most models are trained primarily on public domain data, which rarely include specialized or proprietary DSLs. For this reason, the direct use of these models is not recommended, as it could produce inaccurate or erroneous results, especially when dealing with very specific and less-known DSLs.

This paper addresses this challenge by presenting DSL-Xpert 2.0, an enhanced framework that integrates LLMs for seamless DSL code generation and validation [13]. To this end, the tool allows users to configure models for their specific DSLs through grammar prompting and few-shot learning. In addition, DSL-Xpert 2.0 introduces features such as automatic validation of grammar definitions and input/output correction, ensuring reliable results. Moreover, to increase flexibility, the tool also supports integration with external platforms such as OpenAI [14], HuggingFace [15], and WebLLM [16] for advanced customization. All in all, this results in a flexible and intuitive tool that can speed up development times and reduce complexity, especially helping new users get into a new DSL.

In this context, the main contribution of this work is introducing a straightforward, general-purpose tool that allows applying several LLMs to generate code for a wide variety of DSLs. This tool eliminates the need for users to have technical knowledge, significantly speeding up both the development and learning process. The most remarkable features of the proposed tool are as follows:

- **Broad applicability.** The tool is designed to support a wide range of DSLs, allowing users to work with different grammars and syntaxes with minimal effort.
- **User-centric design.** The interface and functionality prioritize accessibility and ease of use, ensuring that even non-technical users may benefit from its capabilities.
- **Seamless integration.** The tool offers robust integration options for working with multiple LLMs, allowing flexible customization based on the specific requirements of different DSLs.
- **Robust validation mechanisms.** Built-in features ensure that the DSL grammar and generated results meet accuracy and consistency standards.

Additionally, this paper provides a thorough performance analysis, outlining methods to assess the tool's effectiveness across different LLMs. It highlights the tool's adaptability to various scenarios while also pointing out areas for optimization.

The rest of this paper is structured as follows. First, Section 2 provides an overview of related work and fundamental concepts in DSL generation and LLM applications. Section 3 follows, offering a detailed explanation of the design, architecture, and key features of DSL-Xpert 2.0. After that, in Section 4, a representative usage example is shown, with each of the necessary steps to carry it out. Then, in Section 5, a thorough evaluation framework is presented, including the models and hyperparameters tested, followed by a discussion of the evaluation results, highlighting the strengths and weaknesses of the tool in various contexts. Following that, Section 6 presents a short user study to assess the perceived usefulness and acceptance of DSL-Xpert 2.0. Finally, Section 7 concludes with a discussion of the known limitations and potential future research directions, emphasizing how these can guide the next steps for both the tool and the field of DSL generation.

## 2. Background

This section reviews existing research on automated DSL-specific code generation, highlighting key concepts that guide tools like the one proposed in this paper. Although the use of LLMs for DSL-related tasks is still an emerging field, there have been several studies that provide diverse approaches. These include simple uses of GPT-based models for DSL tasks [17], as well as studies on how various LLMs, prompt designs, and DSL types impact performance [18,19]. The tool presented in this paper, DSL-Xpert 2.0, builds on these previous advances, integrating insights from prior works to create an efficient and easy-to-use solution.

With the widespread use of LLMs in recent years, the semantic parsing field is being widely explored [20]. This process involves interpreting the meaning behind linguistic expressions, enabling LLMs to convert natural language instructions into structured representations. For DSLs, this means that when a user provides an instruction such as "create a new product", the LLM must be able to translate it into DSL-specific syntax rather than performing the literal action. Modern LLMs, such as OpenAI's GPT [14] or Meta's LLaMa [21], have been trained with large amounts of data, making them ideally suited for this task. Developing a custom model to achieve similar results would require enormous resources, making the use of these pre-trained models a more practical and efficient solution.

Furthermore, to guide LLMs to produce correct results when generating specific DSL code, it is crucial to define the rules governing the syntax and semantics of the DSL. In many cases, pre-trained models are already capable of performing general-purpose tasks without the need for specific guidance. However, DSL code generation requires an explicit definition of its vocabulary to ensure accuracy. The process of feeding this vocabulary to an LLM is known as grammar prompting [22]. A common approach to defining these rules is using the Backus-Naur form (BNF) [23,24], as it provides a clear and intuitive structure. Other formats, such as JSON or XML, can also be effectively used to represent the DSL grammar [25,26]. In any case, by incorporating these predefined grammar structures, LLMs can ensure that their responses conform to the intended DSL format, improving the quality of the generated code.

Another technique that improves LLM performance is few-shot learning, in which the model is provided with a limited set of representative examples to infer patterns and generate new outputs [27]. In this context, each example consists of a natural language instruction paired with its corresponding DSL representation. The more diverse and extensive the examples are, the greater the model's generalization capabilities [28]. However, this approach has some limitations, as the accuracy of the model is highly dependent on the examples provided. If the model encounters instructions that differ significantly from the examples it has seen, its ability to produce correct answers may be compromised. Despite these challenges, few-shot learning is a fast and efficient method for guiding LLM behavior, making it a valuable tool for DSL code generation.

Although pre-trained LLMs work well for many general tasks, they can have problems when dealing with specialized use cases, such as the ones pointed out in Section 1. To overcome this issue, fine-tuning is a common strategy in which a model is trained with domain-specific data to refine its knowledge [29,30]. However, fine-tuning is not the only option. A complementary and increasingly popular technique is retrieval-augmented generation (RAG) [31], which supplements the model's responses with external information rather than retraining it from scratch. Although these approaches offer significant advantages, they require serious computing resources and access to large, high-quality datasets, making them more resource-intensive when compared to methods such as grammar prompting or few-shot learning. For this reason, neither RAG or fine-tuning have been directly integrated into the tool, although the models resulting from those methods (applied externally) can still be used. In any case, these techniques remain an

effective method for improving LLM performance on specialized tasks such as DSL code generation.

Although pre-trained LLMs work well for many general tasks, they can struggle with specialized use cases, as discussed in Section 1. One strategy to address this limitation is fine-tuning, where the model is retrained using domain-specific data to improve performance in targeted scenarios [29,30]. However, fine-tuning is not the only option. A complementary and increasingly popular approach is retrieval-augmented generation (RAG) [31], which improves the model's responses by dynamically incorporating relevant external information at inference time, rather than modifying the model's parameters. While RAG can achieve comparable or even superior results to fine-tuning in some contexts, especially for knowledge-heavy tasks like DSL code generation, it also demands significant computational resources and high-quality retrieval infrastructure. For this reason, RAG and other resource-intensive adaptation methods have not been directly integrated into the tool, although models or systems using them externally can still be employed.

Nonetheless, in the past years, research on LLM-based DSL code generation has accelerated, resulting in innovative approaches and insights. Most researchers have focused on comparing different alternatives in search of the best ones for the DSLs they used [32–34]. The results show how combining different methods, such as few-shot learning and fine-tuning, culminating in an RAG implementation, seems to be one of the best alternatives. However, the use of this kind of approach, as mentioned above, implies the need for an enormous amount of resources, which is not feasible for a general-purpose application. On the other hand, other works have also explored more definite concepts of the code generation process, such as result validation, offering a concrete framework with which to carry it out [35]. Still, the number of papers that attempt to overcome the problems of working with DSLs using LLMs is minuscule. One of the few we have found is [36], which presents a tool capable of generating a DSL grammar from natural language. However, this only addresses a specific case within the natural flow of DSL creation and maintenance. Therefore, while equally useful, it cannot be directly compared to the tool presented in this paper. In any case, this rapid progression underscores the growing importance of tools that simplify the application of these methods for non-expert users. DSL-Xpert 2.0 builds on this progress, aiming to bridge the gap between cutting-edge research and practical application by providing a simple, user-friendly solution that incorporates these advances to make DSL-related tasks more accessible.

### 3. Software description

This section is focused on showing in detail the entire proposed tool. For this purpose, in Section 3.1, each of the functionalities provided in the tool, as well as their advantages and limitations, are discussed in depth. Subsequently, Section 3.2 presents the architecture of the tool, with a brief description of each of its components and how they relate to each other.

#### 3.1. Software features

DSL-Xpert 2.0 presents functionalities that make handling any DSL and communication with its corresponding LLM much more comfortable, flexible, and reliable for any potential user. This section will discuss these main features in detail, starting with the model creation and the communication interfaces between DSL and LLM, in Section 3.1.1. Next, in Section 3.1.2, the validation process of grammars and usage examples of the specified DSL, as well as the responses provided by the configured LLM, will be highlighted. Finally, Section 3.1.3 will list the various third-party integrations performed and why they are relevant to the tool.

##### 3.1.1. Model setup and interaction framework

DSL-Xpert 2.0 helps domain experts work with different DSLs and LLM models. The tool simplifies defining DSLs, selecting models and specifying specific settings to address different user needs. This feature set ensures that the tool can adapt to a wide variety of contexts and use cases with minimal effort, promoting usability and accuracy in DSL-related workflows.

One of the key features is the ability to define and configure custom models adjusted to the specific needs of each DSL. Users can specify the DSL grammar, provide usage examples, and set other parameters to align the LLM's behavior with the desired domain. Notably, these configurations are independent, allowing users to modify independent definition model aspects. As illustrated in Fig. 1, when creating a model, the user has to cover the following sections to create it:

1. **Model Info.** The user enters information related to the name of the model and a brief description of its purpose.
2. **Model Setup.** The user selects the LLM configuration, such as the *Model Type* (which can be OpenAI, HuggingFace, WebLLM or other integrations), the model to be used (depending on the model type it will be different) and extra parameters that might be required depending on the model (such as an API KEY).
3. **Model Hyper Parameters.** The user selects hyperparameters related to the model, since they are independent of the selected model. Here, configurations related to temperature, top P, and other parameters can be defined so that the models behave in the desired way. In addition, the tool notifies users if a recommended set of hyperparameters is available for the selected model, based on those discussed in Section 5.
4. **Grammar Setup.** The user defines both grammar and input/output examples of DSL definitions that will be passed as context to the LLM.

All configurations are stored in a database, allowing users to switch between projects or resume work easily by selecting the corresponding project card, as shown in Fig. 2. This modular design offers significant flexibility for managing multiple DSLs or addressing diverse problems within a unified environment. For example, users can create one model optimized for generating DSL code and another designed to assist newcomers in understanding the syntax and structure of a DSL.

Once a model is configured, DSL-Xpert 2.0 provides a chat-based interface that establishes real-time interaction between the user and the selected LLM. This interface ensures that the natural language instructions entered by the user are quickly translated into accurate DSL results. The chat also makes it easy to refine the DSL code iteratively, letting users adjust inputs and instantly see the resulting changes in the output. Fig. 3 illustrates the intuitive design of the tool, showing the defined model parameters and the interaction process.

##### 3.1.2. Validation mechanisms

As mentioned previously, one of the key objectives of DSL-Xpert 2.0 is to eliminate obstacles for users with limited experience in the target domain. For this purpose, several specialized features have been implemented, with grammatical validation being one of the most notable. This feature allows users to verify the grammar definition of the provided DSL almost effortlessly, without requiring extensive domain knowledge. That is made possible through Langium [37], which was chosen for its novelty and ongoing use in the scientific community, as well as its relatively straightforward integration with the tool [38,39]. Consequently, when defining a new model in the tool, users can access the editor that provides immediate feedback on grammar validity, as illustrated in Fig. 4.

After validating the grammar, the user can incorporate usage examples into the model, enhancing its context and potentially improving its accuracy. Each example consists of two components: the input, which is written in natural language and represents the instruction or the desired

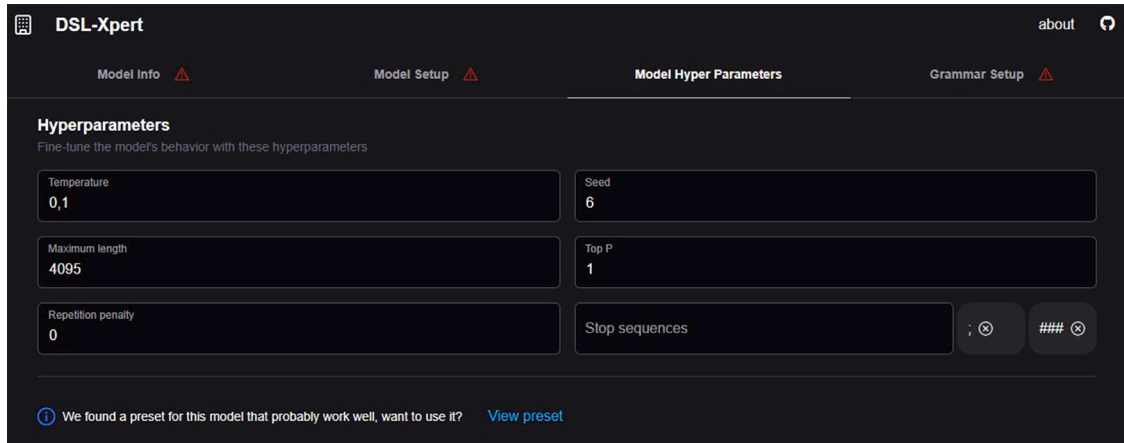


Fig. 1. Model creation process, highlighting its independent definitions.

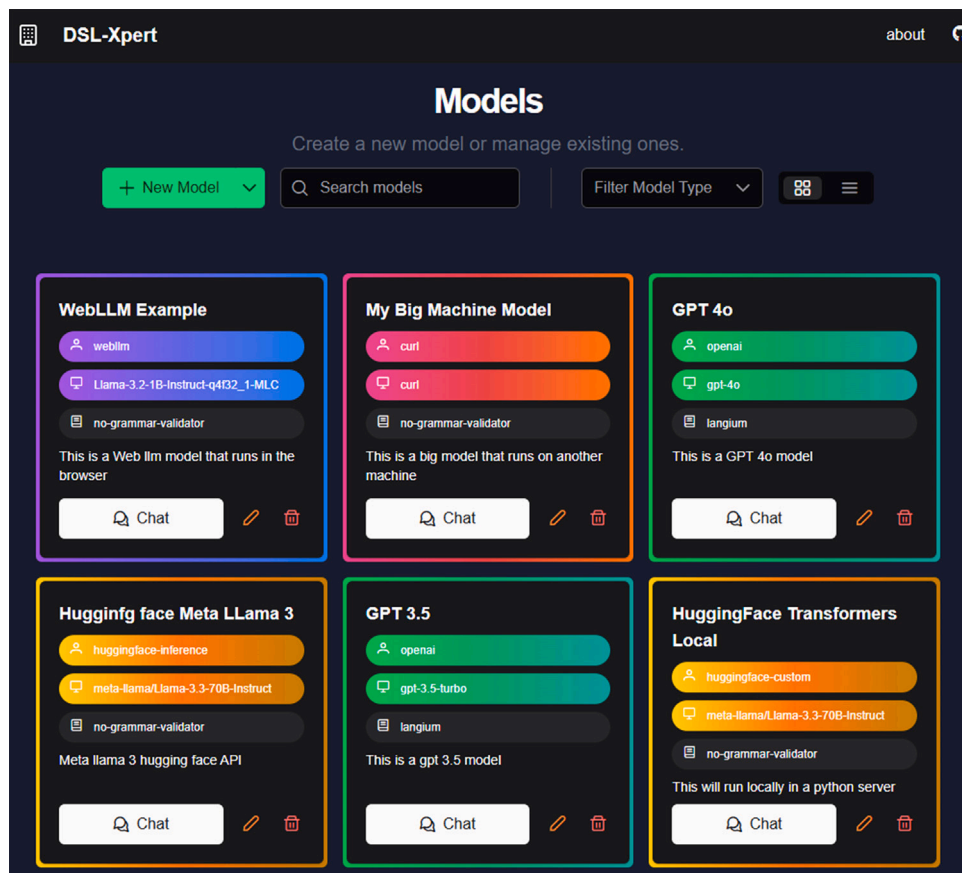


Fig. 2. DSL-Xpert 2.0's home interface, showcasing different model cards.

translation into the target DSL; and the output, which reflects the expected response from the model. Similarly to the definition of the DSL grammar, these examples are also validated against the DSL grammar to ensure compliance. This process guarantees that the provided examples align with the specified DSL structure.

Moreover, the user has the option to enable or disable grammatical validation in the final model. If enabled, while interacting with the model through the chat interface, the user can verify whether the

responses generated by the LLM comply with the defined grammar. Furthermore, they can access detailed information to identify specific errors, as illustrated in Fig. 5.

### 3.1.3. Third-party integrations

In order to provide users with maximum flexibility and ease of use, several integrations have been implemented, allowing for more precise tuning of the target domain. That also enables users to choose the

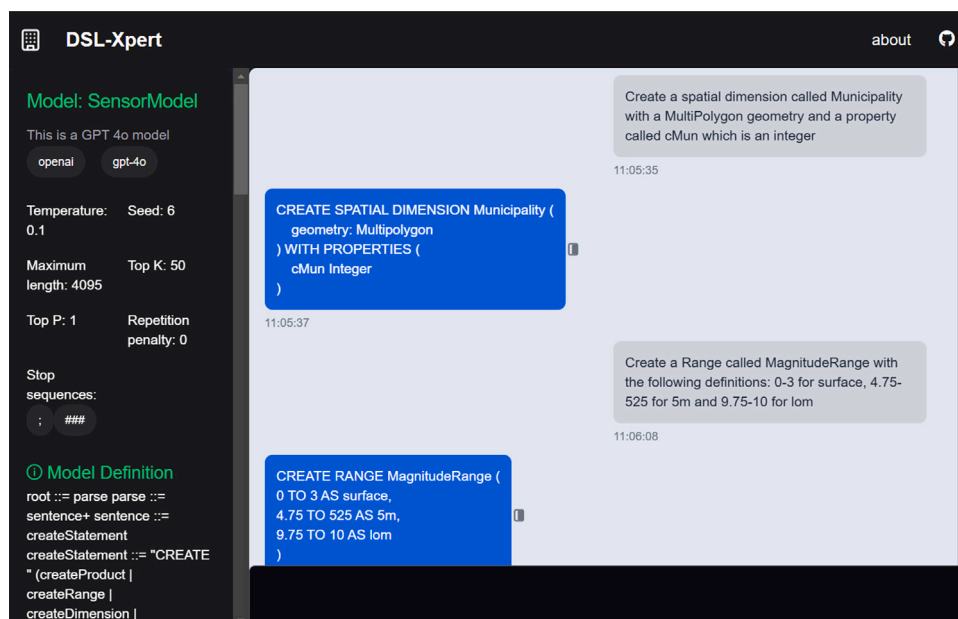
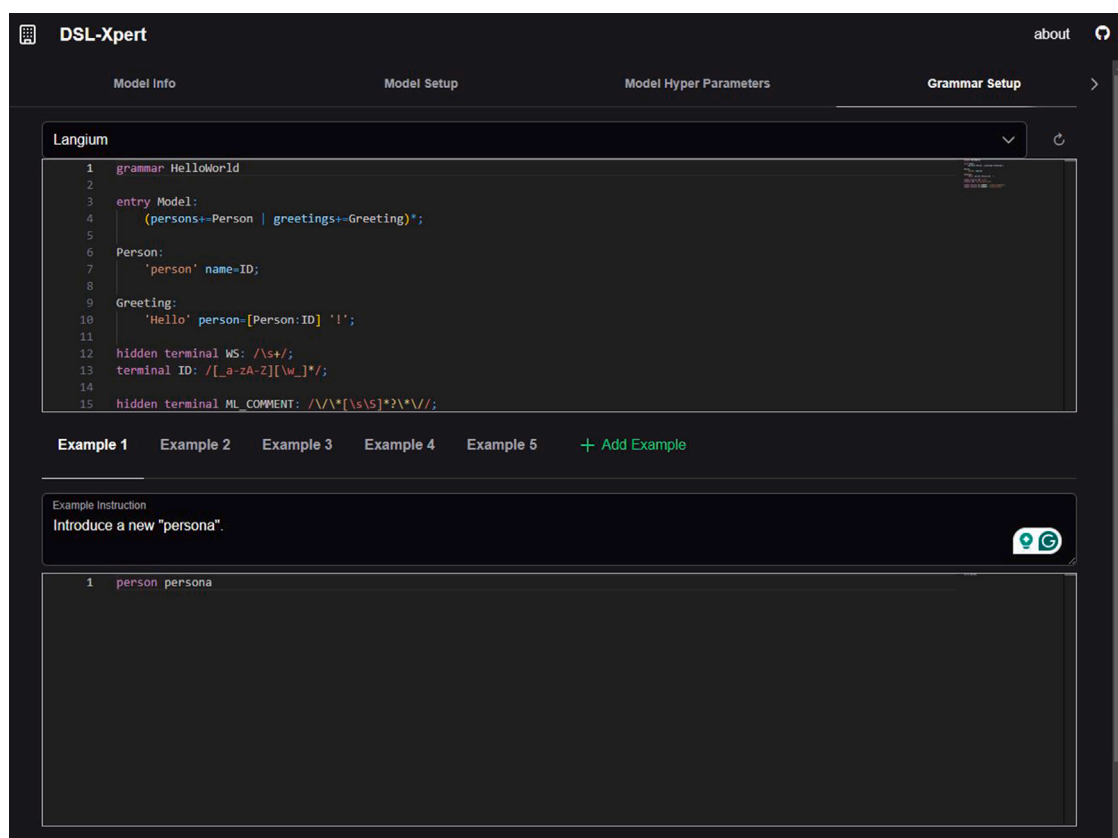


Fig. 3. DSL-Xpert 2.0’s chat interface, showing the different parameters of the “SensorModel” model, along with some response examples from the selected LLM.



**Fig. 4.** Grammar validation example while creating or editing a model.

features they wish to use based on their preferences and domain knowledge. In addition, the model's type selection or integration is entirely independent of the other features defined in the model, meaning users can experiment with different options without altering the other model definition aspects, as highlighted in Section 3.1.1.

The built-in implementations of the tool are described in detail below:

- **OpenAI.** Currently, OpenAI provides some of the most widely used models worldwide, delivering high accuracy in various



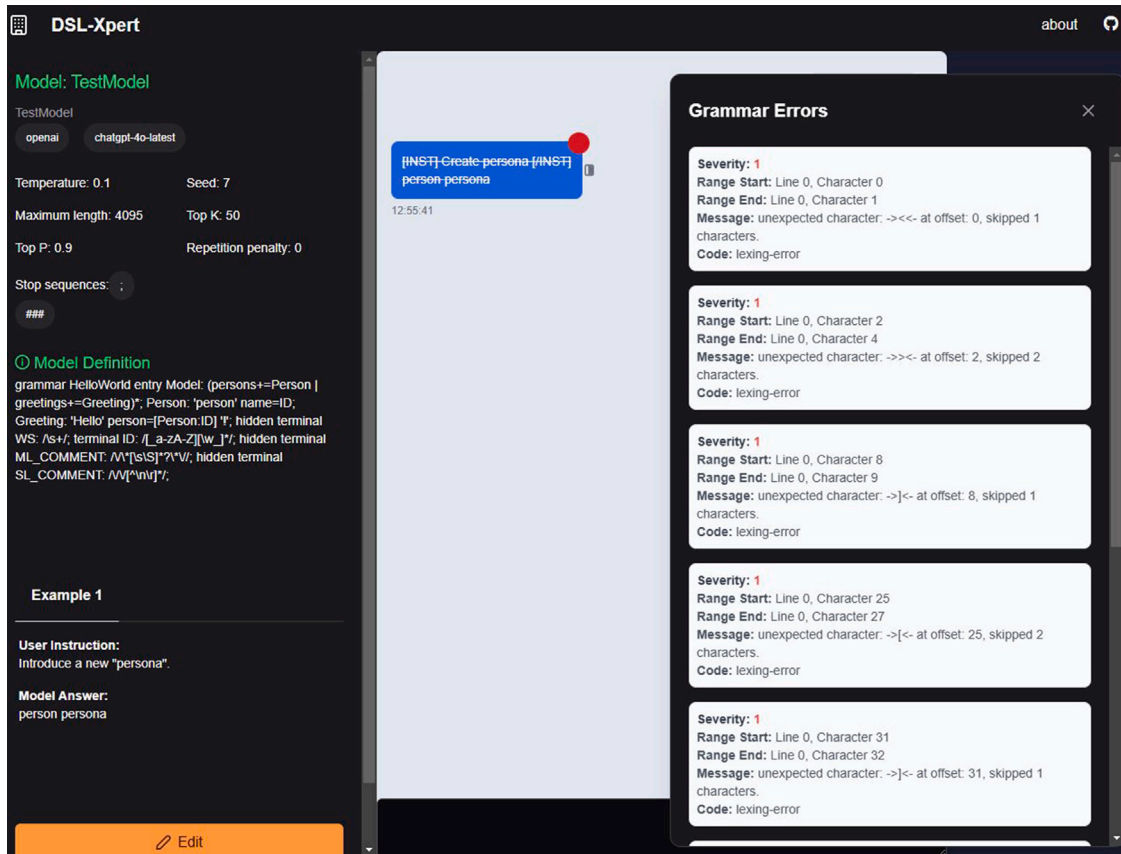


Fig. 5. Chat validation error example, along with all its details.

tasks, including those relevant to the proposed tool. Given its widespread adoption, it is a natural option to include. To utilize OpenAI models, users only need an API key (available for a fee) to connect the model to the tool.

- **HuggingFace.** HuggingFace hosts the largest repository of LLMs, enabling users to download pre-trained models for their specific tasks. Due to its extensive offerings, it is an essential integration in the proposed tool. DSL-Xpert 2.0 provides two options for utilizing HuggingFace models:
  - **Inference.** Some of HuggingFace’s more powerful models can be run on their servers, although the number of available models in this option is significantly smaller than the entire repository. Using this feature requires an API key to authenticate users accessing these resources.
  - **Transformers.** This option allows users to run any model available in the HuggingFace repository locally. Although it offers flexibility, it requires local computational resources to run the models.
- **WebLLM.** WebLLM enables users to load models directly in their web browser without the need for server deployment. By using WebGPU, models can take advantage of GPU resources on the client side. However, a key limitation is that the models available through WebLLM are relatively small and may not be suitable for complex tasks. However, they provide a quick and convenient solution for simpler use cases.
- **Custom REST API.** This integration allows users to connect to any specified host and port, simplifying the use of models deployed on external machines. As long as the external machine exposes a REST API, users can access the models remotely. To ensure compatibility and maintain a consistent structure with OpenAI, all API requests must adhere to the OpenAI standard.

The selected integrations are designed to meet different user requirements. First, users can leverage their models externally or locally via the Custom REST API or HuggingFace Transformers. Secondly, they can also utilize well-established models from the industry and scientific community through OpenAI and HuggingFace Inference. Finally, WebLLM provides an interesting solution for quick and easy access to simpler models.

### 3.2. Software architecture

The tool’s architecture is primarily based on the client–server model, which ensures scalability and efficient communication between its components. Fig. 6 provides an overview of the general architecture of DSL-Xpert 2.0, illustrating all the components that make up the system. Furthermore, the tool is fully containerized using Docker [40] and Docker Compose [41], allowing seamless deployment and execution.

The client, built using Vue.js [42], provides a set of interfaces that allow end users to view and search for previously created models using the `ModelList.vue` component. In addition, users can define new models or change previously defined ones using the `Form.vue` component, which includes grammatical validation for the DSL definition and the instance examples. This is handled using Langium, leveraging web workers to offload processing from the main JavaScript execution thread (`langium-lsp.js` and `grammar-lsp.js`), ensuring a smooth and uninterrupted user experience.

Moreover, users can interact with models via chat, using the `Chat.vue` component. This feature employs `contextGenerator.js`, which is responsible for generating a suitable context to be sent to the various LLM integrations. To do so, it uses as information all the messages generated in the chat, the definition of the DSL, the examples of inputs/desired outputs and other needed information. In addition, if it has been specified that the model has a grammatical validation,

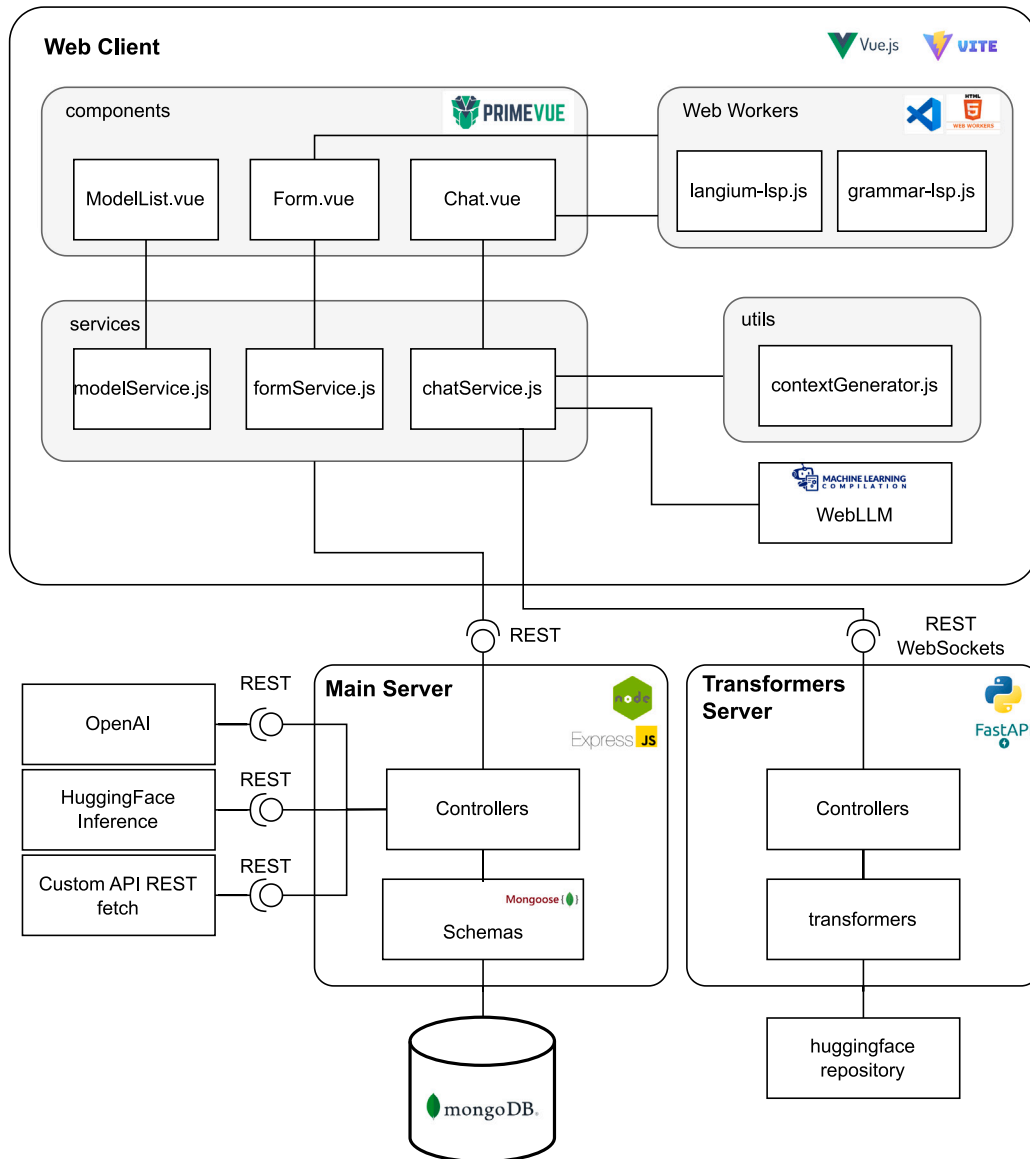


Fig. 6. DSL-Xpert 2.0's architecture.

the chat will also use the **WebWorkers** to validate the responses that the LLM models return. Furthermore, if the defined model is of the **WebLLM** type, the chat component uses the **WebLLM** library to execute the model on the client side.

The system includes two distinct servers. First, the **Main Server**, built with Node.js [43], exposes a REST API that is used by the **Web Client**. In addition to that, it is also responsible for managing database connections using **MongoDB** [44], through the **Mongoose** library [45], to maintain information related to the models. On the one hand, this server stores the information of the models defined by the user, such as the name, the description, the hyperparameters it needs, the type of models they are, the definition of the grammar, the examples of input/desired outputs and any other information that the user has defined in the model form. In this way, it is guaranteed that the defined models contain all the required fields. On the other hand, this server has integrations with various AI providers, including **OpenAI**, **Hugging Face**, and a custom REST API that supports **OpenAI**-compatible requests. When a user makes a request using the chat, the server is responsible for sending it to the correct service.

Secondly, the **Transformers Server**, developed in Python, enables users to use their own models in their local machine using the

**Transformers** library [46]. The only prerequisites are that the model must be hosted on **Hugging Face** (whether publicly or privately) and be compatible with the **Transformers** framework. In addition, communication with the client is handled via web sockets, providing real-time feedback on the model's loading process.

#### 4. Running example

To illustrate the system's capabilities, this section presents a practical example of usage in the domain of geoscientific Internet of Things (IoT). This domain typically involves the collection and analysis of large volumes of environmental sensor data, such as temperature, humidity, and air quality, captured over time and space. The goal is to structure these data in a way that supports storage, visualization, and analytical querying within an IoT data warehouse.

This example builds upon the low-code framework for IoT data modeling proposed by Lamas et al. [47], which is specifically designed to address the challenges of geoscientific data, such as heterogeneous sensor deployments, spatial hierarchies, and time-series data integration. This framework proposes a DSL that enables users to describe the structure of an IoT sensor network and its corresponding data

model, allowing users (especially those without extensive programming experience) to define key components of an IoT data warehouse. These components are defined as follows.

**Sensors** represent logical groupings of devices that measure specific environmental or operational phenomena. Each sensor is described by attributes such as its name, measurement interval, geometry type (such as point or polygon), and whether it is stationary or mobile. Additional custom properties, such as device ID or installation date, can also be defined. In addition, these sensors produce **measurements**, which are the variables that they record over time (such as temperature or pressure). Measurements are described with metadata, including their data type, units, and optional visual elements such as icons. Besides, they may also include range definitions to support validation and visualization. Furthermore, to structure and analyze these data, the model incorporates **spatial and categorical dimensions**. Spatial dimensions can capture hierarchical relationships, for example, sensors located within neighborhoods that are part of cities, enabling geographic aggregation. Categorical dimensions, on the other hand, group data by non-spatial features such as device type or data source. Apart from that, the model also incorporates temporal dimensions, derived from the sensor's data collection intervals, to support time-based queries, such as daily averages or long-term trends. Finally, sensors can be organized into **SensorGroups** for logical classification, and an entire sensor network project is encapsulated within a **Product**, which includes identifying information such as a name and a spatial reference system identifier (SRID) to provide geographic context.

At this stage, the researcher configures DSL-Xpert 2.0 using the defined grammar, which captures the structure and semantics of their specific IoT data model. Once the grammar is in place, the tool enables researchers to generate valid DSL instances by simply describing them in natural language through the chat interface. This interaction is made possible through a series of configuration steps within DSL-Xpert 2.0, starting with the definition of the model itself. To do so, the user provides a name and a short description of the IoT data model. As shown in Fig. 7, this step helps establish the core identity of the model and sets the foundation for the following setup steps.

Following that, the user configures the characteristics of the LLM environment. As shown in Fig. 8, this involves selecting an LLM provider, such as OpenAI, HuggingFace, or WebLLM (or even their very own one through a custom REST API), as well as entering any required API credentials and access configurations. Additionally, the user can adjust the model parameters, such as temperature and top p, to control the creativity and sampling diversity of the language model, as shown in Fig. 9.

Once the environment is configured, the next step is to define the DSL grammar that represents the structure of the IoT data model. As shown in Fig. 10, the user writes the grammar in Langium format, accompanied by example instruction-instance pairs that illustrate the intended use of the DSL. Although alternative formats such as Backus–Naur Form (BNF) or ANTLR may also be used for grammar definition, validation is only supported for Langium in this version. These examples provide contextual input during interactions with the language model. DSL-Xpert 2.0 performs automatic validation to ensure that grammar and examples are syntactically and semantically correct before proceeding.

With the grammar and examples in place, the user can begin to interact with the system through the DSL-Xpert 2.0 chat interface. By entering natural language prompts, the user can request new DSL definitions that match the configured grammar. For instance, Fig. 11 illustrates a detailed prompt that describes a sensor system with various properties and spatial dimensions. DSL-Xpert 2.0 parses this prompt, combines it with the predefined grammar and examples, and uses the LLM to generate a corresponding DSL instance that is both valid and contextually aligned.

This workflow allows users, even those without expertise in the specific DSL, to define rich and semantically meaningful IoT sensor systems using natural language. In doing so, it significantly simplifies the process of building complex, spatially aware data warehouse models.

Fig. 7. Running Example: IoT sensors - name and description.

Fig. 8. Running Example: IoT sensors - model characteristics.

## 5. Evaluation

This section presents a thorough evaluation of the proposed text-to-DSL tool, using various LLMs. The analysis focuses on identifying the LLMs and parameter configurations that achieve the highest accuracy,



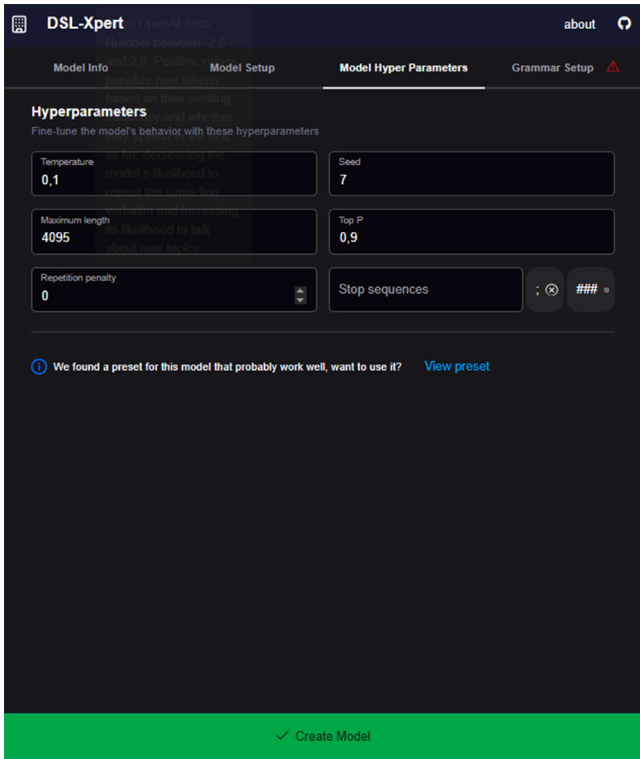


Fig. 9. Running Example: IoT sensors - model hyperparameters.

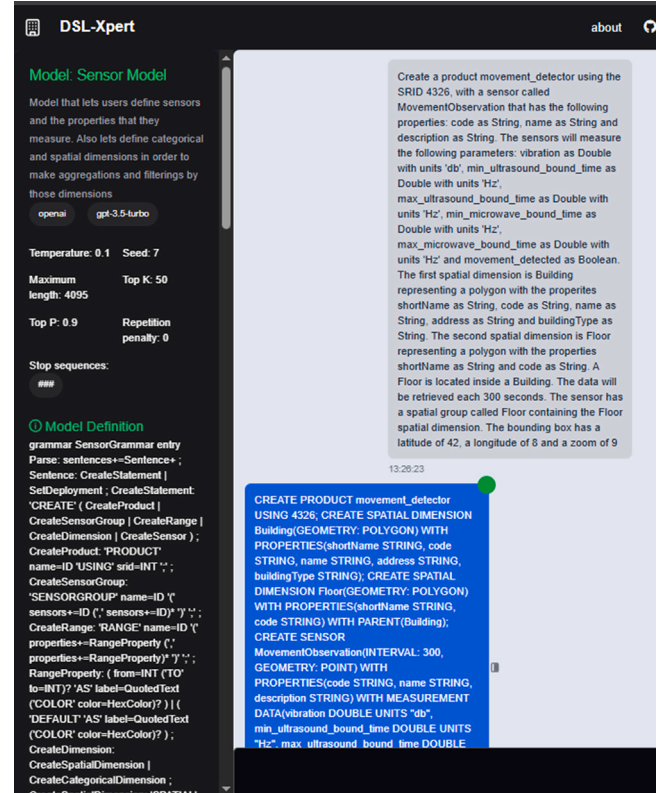


Fig. 11. Running Example: Chat and prompt example.

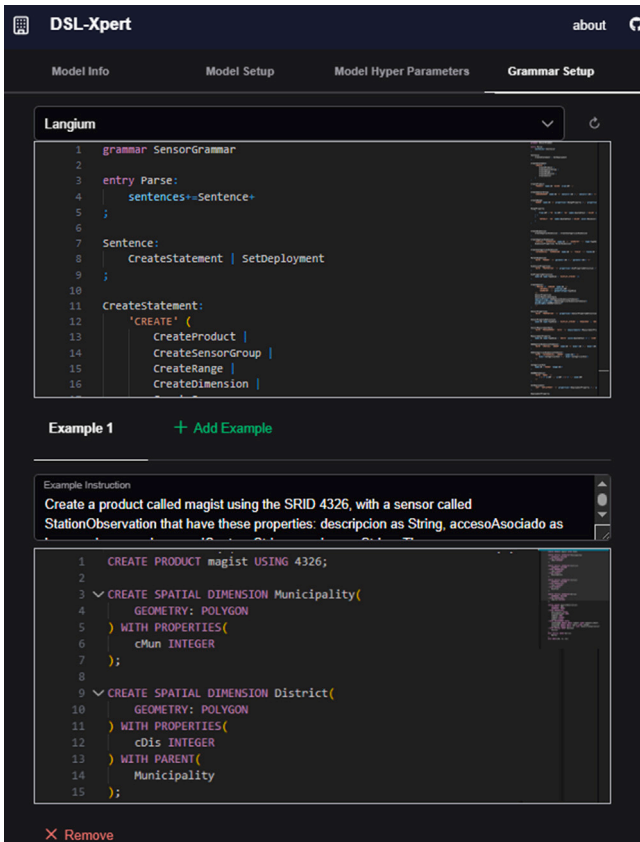


Fig. 10. Running Example: Model grammar and instruction-instance examples.

complemented by insights from BLEU score evaluations (Bilingual Evaluation Understudy, [48]). Different levels of quantization and model dimensions are also examined to enable a detailed comparison of the tested approaches.

Moreover, it should be noted that all evaluations were carried out using the same core techniques implemented in DSL-Xpert 2.0: structured context generation, prompt design, and grammar-based validation. Although the experiments were run outside of the tool's interactive interface for practical reasons, they relied entirely on the internal logic and validation steps of DSL-Xpert 2.0. As a result, the outcomes presented in this section not only reflect the performance of the language models, but also demonstrate the effectiveness of the tool design in guiding and validating model outputs in realistic scenarios.

Section 5.1 outlines the experimental framework, detailing the handcrafted dataset, the DSLs utilized, and the diverse LLM configurations (e.g., model size, quantization levels). The metrics employed for evaluation accuracy and BLEU score are introduced and justified in Section 5.2. The results, presented in Section 5.3, include an in-depth discussion of key findings, particularly the observation that smaller, open-source models can outperform much larger ones. Finally, Section 5.4 concludes with general remarks on practical implications, summarizing the whole experimentation.

### 5.1. Experimental setup

The experimental setup for this evaluation is structured into four distinct components: the dataset and test examples, the evaluated models, the prompting and grammar configurations, and the parameters considered. In the following paragraphs, these components are described in detail, explaining their relevance and how they contribute to the overall assessment.

You are a helpful assistant. You are tasked with generating Domain Specific Language (DSL) code for a given input. Respond only with the DSL code.

Fig. 12. The standardized system prompt used for evaluating DSL generation.

**Dataset and test examples.** The dataset consists of 28 handcrafted text-to-DSL examples. Although it is small in size, the dataset is designed to cover a wide range of complexities, with a focus on high-quality, well-defined scenarios. This design ensures that it effectively tests the models' performance across various difficulty levels, making it an ideal foundation for capturing the nuances of model behavior. In addition, these DSL examples were collaboratively constructed and reviewed by different authors to ensure structural variety, domain relevance, and coverage of both simple and complex DSL constructs. No post-hoc exclusion was applied based on performance.

**Evaluated models.** Extensive testing has been carried out on several LLMs, including both general-purpose versions and those designed for coding jobs. The models that were tested included:

- **Closed-source models:** OpenAI GPT-4o [49] and GPT-4o mini [50], utilized as reference baselines for high-quality performance.
- **Open-source models:** StarCoder2 (3B, 7B, 15B) [51], QwQ (32B) [52], Qwen 2.5 (7B, 32B, 72B) [53], LLaMA 3.1 variants (8B, 70B) and 3.3 (70B) [10], CodeLlama (7B, 13B, 34B, 70B) [54], Falcon3 (7B, 10B) [55], Mistral (7B) [56], Mistral Large 2 (123B) [57], Mistral Nemo (12B) [58] and Nemotron (70B) [59].

The study examined models of different sizes (from 3B to 123B) and quantization levels (4-bit, 6-bit, 8-bit, and full precision). Quantization is a widely used technique to reduce the bit-precision of deep learning models, significantly reducing model size and accelerating inference [60–65]. These settings revealed insights into the trade-offs between memory efficiency and performance, allowing for a thorough examination of their effects on accuracy and BLEU scores.

**Prompting and grammar setup.** All models were tested with a common prompt that ensured uniform and precise DSL creation. This prompt was designed to direct the models towards generating DSL code, as shown in Fig. 12. It included a simple command to limit the output to DSL code, the DSL grammar, and four illustrated few-shot examples that were carefully chosen to guide the model while avoiding data leaks.

To cut down on computing costs, once the system prompt was finalized during the initial trials, extensive experimentation with different prompts across all models was avoided. This approach struck a balance between refining the prompt and ensuring efficient use of resources.

**Parameters considered.** To find the best setup for the text-to-DSL task, eight alternative combinations of the parameters `temperature`, `top_p`, and `top_k` were evaluated. In order to achieve a balance between regulated creativity and targeted production, these features were selected. In particular:

- **Temperature.** To minimize randomness, the value 0 has been used ensuring deterministic and consistent outputs, an essential factor for the precision demanded by text-to-DSL tasks. Conversely, a higher value like 0.5 was introduced to explore slight variability.
- **Top\_p.** This parameter was set to 0.8 and 0.9 to control the cumulative probability of token selection, effectively narrowing down the token choices to the most probable ones, and ensuring relevant and high-quality outputs.
- **Top\_k.** With values such as 30 and 50, the model was restricted to selecting from a smaller set of likely next tokens, minimizing the chances of generating irrelevant or off-topic responses.

Since the objective of the text-to-DSL task is to generate precise and organized DSL code, little creativity was needed. Clarity, accuracy, and conformity to the desired output format were therefore given top priority in the combinations that were chosen.

## 5.2. Evaluation metrics

Two key metrics were selected to evaluate the performance of each model: accuracy and BLEU score. These metrics were chosen for their ability to assess both the precision and overall quality of the generated DSL code, which is central to the goal of this task.

**Accuracy.** Accuracy served as the primary metric and was evaluated by executing the manually crafted query (gold standard) alongside the query generated by the LLM. This approach allowed us to determine the actual executability of the queries, rather than relying solely on a perfect match comparison. By focusing on execution, we avoided discarding potentially valid queries that may have differed slightly from the gold standard due to minor, non-critical variations.

**BLEU score.** The BLEU (Bilingual Evaluation Understudy) score is a widely used metric to assess the quality of generated code by comparing it against human-provided reference texts [48].

These studies use BLEU to evaluate the models' ability to produce SQL queries that are both syntactically and semantically similar to the references. The use of BLEU in these contexts is motivated by the need for a quantitative metric that reflects the quality of the generated outputs compared to an ideal solution. While BLEU does not ensure the executability or semantic correctness of the generated queries, it provides a valuable measure of structural and lexical fidelity to the reference.

In our case, we adopted BLEU to assess the quality of text-to-DSL conversions. This metric, widely used in evaluating code generation tasks [54], allows us to quantify the similarity between the model-generated DSL code and the reference code, helping identify structural or lexical differences that may affect final correctness. Despite its limitations, BLEU offers a standardized way to compare model performance and track improvements over time.

## 5.3. Results and analysis

The accuracy and BLEU score of the assessed models are summarized in Table 1. With 100% accuracy in all 28 samples, OpenAI GPT-4o was the most robust baseline. High BLEU scores and reliable, flawless performance make GPT-4o the “gold standard” for the text-to-DSL task.

Several open-source models produced competitive results despite GPT-4o's dominance, especially when taking into account their smaller parameter counts and the viability of local implementation. The GPT-4o mini variant's accuracy of 71.4% was surpassed by the accuracy of 82.1% attained by StarCoder2 (7B).

Among open-source models, StarCoder2 (7B) emerged as the best performer in simple cases, achieving an accuracy of 94.4%. This suggests that StarCoder2 is highly effective when handling well-structured and straightforward transformations. However, its accuracy drops significantly to 60% on complex examples, revealing its limitations in handling more intricate text-to-DSL conversions. In contrast, QwQ (32B) and other larger models maintained better consistency across difficulty levels, demonstrating higher robustness for complex transformations. These results highlight how smaller, open-source models, when correctly adjusted and aligned, can perform on par with or better than bigger, closed-source models, particularly in environments where resources are limited or privacy is a concern.

However, some larger models, such as Qwen 2.5 at 72B, did not outperform StarCoder2 (7B), suggesting that raw model size does not always guarantee better results. Factors such as alignment techniques, prompt sensitivity, and pretraining biases likely play a significant role.

**Table 1**

Performance results for text-to-DSL tasks across various LLMs. The table includes columns detailing the model name (with FP indicating full precision and quantization levels specified as 4, 6, or 8 bits), average accuracy, accuracy on complex examples, average BLEU score, BLEU score on complex examples, BLEU score on simple examples, and the total number of parameters.

| Models                     | Acc_avg | Acc_com | Acc_sim | Bleu_avg | Bleu_com | Bleu_sim | Parameters                    |
|----------------------------|---------|---------|---------|----------|----------|----------|-------------------------------|
| OpenAI GPT4o               | 1       | 1       | 1       | 0.959    | 0.994    | 0.939    | top_p: 0.8, t: 0              |
| StarCoder2 (7B) fp         | 0.821   | 0.6     | 0.944   | 0.916    | 0.855    | 0.95     | top_k: 50, top_p: 0.8, t: 0   |
| qwq (32B) 4bit             | 0.786   | 0.7     | 0.833   | 0.928    | 0.944    | 0.918    | top_k: 50, top_p: 0.8, t: 0   |
| qwq (32B) 8bit             | 0.786   | 0.7     | 0.833   | 0.918    | 0.945    | 0.903    | top_k: 50, top_p: 0.8, t: 0.5 |
| qwq (32B) fp               | 0.786   | 0.7     | 0.833   | 0.915    | 0.954    | 0.893    | top_k: 50, top_p: 0.8, t: 0   |
| Qwen 2.5 (32B) fp          | 0.75    | 0.6     | 0.833   | 0.923    | 0.949    | 0.908    | top_k: 50, top_p: 0.9, t: 0.5 |
| Qwen 2.5 (72B) 8bit        | 0.75    | 0.6     | 0.833   | 0.898    | 0.904    | 0.894    | top_k: 50, top_p: 0.8, t: 0   |
| Qwen 2.5-coder (32B) fp    | 0.75    | 0.7     | 0.778   | 0.897    | 0.955    | 0.865    | top_k: 50, top_p: 0.9, t: 0.5 |
| Qwen 2.5 (72B) 4bit        | 0.75    | 0.6     | 0.833   | 0.892    | 0.907    | 0.883    | top_k: 50, top_p: 0.9, t: 0   |
| OpenAI GPT4o mini          | 0.714   | 0.4     | 0.889   | 0.975    | 0.967    | 0.979    | top_p: 0.8, t: 0              |
| LLaMA 3.1 (70B) 8bit       | 0.679   | 0.6     | 0.722   | 0.895    | 0.978    | 0.849    | top_k: 50, top_p: 0.9, t: 0   |
| LLaMA 3.1 (70B) 4bit       | 0.679   | 0.6     | 0.722   | 0.879    | 0.971    | 0.828    | top_k: 50, top_p: 0.8, t: 0.5 |
| Deepseek r1 (70B) fp       | 0.679   | 0.7     | 0.667   | 0.671    | 0.73     | 0.637    | top_k: 50, top_p: 0.8, t: 0.5 |
| Falcon3 (10B) fp           | 0.643   | 0.4     | 0.778   | 0.882    | 0.924    | 0.859    | top_k: 50, top_p: 0.8, t: 0   |
| Mistral-large2 (123B) 8bit | 0.643   | 0.5     | 0.722   | 0.344    | 0.369    | 0.33     | top_k: 50, top_p: 0.8, t: 0   |
| LLaMA 3.3 (70B) fp         | 0.571   | 0.1     | 0.833   | 0.923    | 0.952    | 0.907    | top_k: 50, top_p: 0.8, t: 0.5 |
| Qwen 2.5-coder (7B) 4bit   | 0.571   | 0.3     | 0.722   | 0.892    | 0.899    | 0.888    | top_k: 50, top_p: 0.8, t: 0   |
| Nemotron 70B fp            | 0.571   | 0.1     | 0.833   | 0.365    | 0.604    | 0.232    | top_k: 50, top_p: 0.9, t: 0   |
| Mistral-large2 (123B) 6bit | 0.571   | 0.3     | 0.722   | 0.338    | 0.33     | 0.343    | top_k: 50, top_p: 0.8, t: 0.5 |
| Qwen 2.5-coder (7B) fp     | 0.536   | 0.3     | 0.667   | 0.862    | 0.888    | 0.847    | top_k: 50, top_p: 0.9, t: 0   |
| StarCoder2 (3B) fp         | 0.536   | 0.3     | 0.667   | 0.815    | 0.833    | 0.805    | top_k: 50, top_p: 0.8, t: 0   |
| Falcon3 (7B) fp            | 0.464   | 0.1     | 0.667   | 0.763    | 0.701    | 0.797    | top_k: 50, top_p: 0.9, t: 0.5 |
| Mistral Nemo (12B) fp      | 0.464   | 0.3     | 0.556   | 0.672    | 0.708    | 0.652    | top_k: 50, top_p: 0.8, t: 0   |
| StarCoder2 (15B) fp        | 0.464   | 0       | 0.722   | 0.508    | 0.373    | 0.583    | top_k: 50, top_p: 0.8, t: 0.5 |
| Codellama (13B) fp         | 0.429   | 0.2     | 0.556   | 0.748    | 0.865    | 0.683    | top_k: 50, top_p: 0.8, t: 0.5 |
| LLaMA 3.1 (8B) fp          | 0.357   | 0.2     | 0.444   | 0.749    | 0.759    | 0.743    | top_k: 50, top_p: 0.8, t: 0.1 |
| LLaMA 3.1 (8B) 4bit        | 0.321   | 0.2     | 0.389   | 0.771    | 0.853    | 0.725    | top_k: 50, top_p: 0.8, t: 0   |
| Codellama (34B) fp         | 0.321   | 0       | 0.5     | 0.733    | 0.798    | 0.697    | top_k: 50, top_p: 0.8, t: 0   |
| Mistral (7B) fp            | 0.321   | 0       | 0.5     | 0.663    | 0.639    | 0.677    | top_k: 50, top_p: 0.8, t: 0   |
| Codellama (7B) 4bit        | 0.286   | 0       | 0.444   | 0.69     | 0.711    | 0.679    | top_k: 50, top_p: 0.8, t: 0   |
| Codellama (70B) fp         | 0.214   | 0.1     | 0.278   | 0.135    | 0.293    | 0.047    | top_k: 50, top_p: 0.8, t: 0   |

While larger models may benefit from extensive pretraining, smaller models might remain more focused on the specific DSL task, leading to better performance in certain cases.

The findings demonstrate that a large number of open-source LLMs can translate text to DSL with good accuracy, often surpassing 70%. Models like StarCoder2 (7B) and QwQ (32B) show that smaller open-source alternatives can serve as acceptable substitutes, especially in scenarios where cost or data privacy concerns are paramount. Even though GPT-4o remains unbeaten with 100% accuracy, these alternatives still perform competitively.

A limited parameter search, however, revealed consistent patterns across the models:

- **Low temperature (0).** High determinism proved beneficial for text-to-DSL tasks requiring strict syntax adherence.
- **Top-k.** Across all experiments, a value of 50 consistently delivered the best results. This setting effectively constrained the range of possible token choices, maintaining fidelity to the DSL grammar without compromising the diversity of generated outputs.
- **Top-p.** The optimal value for top-p varied between experiments. In 13 out of 32 cases, a value of 0.9 was most effective, providing a broader range of token probabilities and introducing slight variability when required. However, in the majority of experiments (21 out of 32), a value of 0.8 yielded better results, suggesting that tighter constraints on token probabilities were generally more suitable for tasks requiring strict adherence to DSL syntax.

These findings demonstrate the importance of parameter tuning to balance flexibility and precision in text-to-DSL tasks.

#### 5.4. Practical implications

The findings show that environments with limited resources can effectively use LLM-driven DSL creation by implementing specified strategies:

- **Effective prompt design.** A well-designed system prompt, along with the DSL grammar and detailed few-shot examples, provides clear guidance for the model.
- **Parameter optimization for determinism.** Lower temperature settings and constrained token sampling enhance the generation of accurate, syntax-compliant code.
- **Appropriate quantization.** Different quantizations, such as 4-bit or 8-bit, significantly reduce memory usage and inference time, enabling real-time or large-scale deployment with minimal impact on performance.

Despite their strong performance in standard programming tasks, large code-specialized models such as CodeLlama (70B) and StarCoder2 (15B) show clear limitations when handling complex DSLs. These models are primarily trained on large datasets of popular programming languages such as Python, Java, and C++, which follow widely established syntactic and semantic conventions. This heavy exposure appears to introduce a bias towards well-known programming paradigms, making it harder for these models to adapt to the unique structures and grammars that characterize many DSLs.

In contrast, smaller or more generalist models often exhibit greater flexibility in these settings. With fewer built-in assumptions from mainstream languages, they are sometimes better equipped to generate or interpret DSL code that falls outside conventional patterns. This challenges the assumption that larger or more specialized models always deliver better results. When it comes to structurally distinct or niche DSLs, the right choice of model depends not just on size or specialization, but on how well it aligns with the specific demands of the task. Models like StarCoder2 (7B) highlight the potential of smaller and open-source alternatives for DSL-heavy workloads, offering practical advantages in terms of cost, deployment, and independence from proprietary APIs.

Finally, it is worth noting that all results discussed here were produced using the same methodological pipeline implemented in DSL-Xpert 2.0. This includes systematic prompt construction, structured

**Table 2**  
Mapping of survey questions to TAM constructs.

| TAM Construct                | Description                                                                    | Question Codes                 |
|------------------------------|--------------------------------------------------------------------------------|--------------------------------|
| Perceived Usefulness (PU)    | Users' belief that DSL-Xpert 2.0 improves performance, accuracy, or efficiency | B1, B2, B3, B4, B5, B6, C4, C5 |
| Perceived Ease of Use (PEOU) | How effortless and intuitive the tool is                                       | C1, C2, C3                     |
| Attitude Towards Use (ATU)   | Affective evaluation of the tool                                               | C6, C9                         |
| Behavioral Intention (BI)    | Likelihood of adopting or recommending                                         | C7, C8                         |
| External/Experience          | Prior DSL experience for context                                               | A1, A2, A3                     |
| Open Feedback                | Qualitative perceptions                                                        | D1, D2                         |

context generation, and grammar-based validation, key elements that guided and shaped the outcomes presented throughout the evaluation.

## 6. User survey

To evaluate the perceived usefulness and acceptance of DSL-Xpert 2.0, we conducted a user study based on the Technology Acceptance Model (TAM) [66,67]. TAM provides a widely accepted theoretical framework for explaining users' behavioral intention to adopt new technologies through two main constructs: Perceived Usefulness (PU) and Perceived Ease of Use (PEOU). These also influence Attitude Towards Use (ATU) and ultimately Behavioral Intention (BI) to continue using the system.

For this purpose, we designed a 18-item short questionnaire (see [Appendix](#)) to capture user perceptions of DSL-Xpert 2.0. The survey consisted of four different sections:

- **Section A. DSL background:** Three questions (A1-A3) measured participants' prior experience with the DSLs and the domain they belong to. These variables provide contextual information and can moderate perceived usefulness or ease of use.
- **Section B. Specific tool performance:** Six items (B1-B6) evaluated task-specific usefulness, including time saved and correctness of generated code.
- **Section C. General tool evaluation:** Nine items (C1-C9) assessed ease of use, general usefulness, user attitude, and behavioral intention to adopt or recommend the tool.
- **Section D. Open feedback:** Two open-ended questions (D1-D2) captured qualitative comments on strengths and areas for improvement.

Regarding the mapping between these questions and the TAM constructs, [Table 2](#) summarizes them. Note that the number of questions corresponding to PU is higher because items B1, B2, and B3 capture perceived accuracy of the generated code, while B4, B5, and B6 measure perceived efficiency gains (time saved), both for each of the DSLs used. On the other hand, items C4 and C5 are more general-oriented. In any case, both dimensions reflect PU, as they assess whether DSL-Xpert 2.0 helps users achieve their goals more effectively and efficiently.

The survey was completed by five participants, all professional software engineers with varying experience in DSL development. Participants were provided with the Langium grammar for each of the three DSLs, along with a set of representative examples for each. The DSLs used were: (i) a complex and very specific DSL involving GIS-related tasks, (ii) the one corresponding to the running example presented in [Section 4](#), and (iii) a simple DSL representing chess game movements. Each participant completed two parts: first, they manually recreated a specific system (or a set of chess movements) using the Langium playground [68] and verifying syntax and validation. Then, they performed the same tasks using DSL-Xpert 2.0 by defining their

**Table 3**  
Participants' prior DSL experience (Section A).

| Participant | Mean (1-5) |
|-------------|------------|
| DEV-1       | 2.00       |
| DEV-2       | 2.67       |
| DEV-3       | 3.67       |
| DEV-4       | 1.00       |
| DEV-5       | 1.00       |

own configuration and interacting with an LLM to generate and refine the corresponding system. After the session, participants completed the survey. Likert-scale items were scored from 1 (strongly disagree) to 5 (strongly agree), and open-ended responses were analyzed qualitatively to extract key themes.

On the other hand, although the survey focuses on participants' perceptions (PU, PEOU, ATU, BI), Actual Use (AU) of DSL-Xpert 2.0 was ensured through the hands-on tasks completed during the evaluation sessions. Each participant directly interacted with the tool across three DSLs, configuring prompts and refining outputs in real time. This practical component provides contextual evidence of AU and supports the validity of the reported perceptions.

Before analyzing the main TAM constructs, we examined participants' prior experience with DSLs (Section A). As can be seen in [Table 3](#), results revealed heterogeneous backgrounds: while two participants (DEV-2 and DEV-3) reported moderate familiarity with those DSLs and their specific area of expertise, the remaining three (DEV-1, DEV-4, and DEV-5) had limited or no prior experience. This variation contextualizes the TAM findings, as it allows us to assess whether perceived usefulness and ease of use were consistent across different levels of expertise. Remarkably, participants with lower initial familiarity rated the tool's ease of use and usefulness as highly as those with greater experience, suggesting that DSL-Xpert 2.0 effectively lowers the entry barrier to working with DSLs.

In regard to the TAM construct's results, [Table 4](#) presents the average scores and standard deviations (SD) for each of them. Overall, participants reported high perceived usefulness and ease of use, along with positive attitudes and strong behavioral intention to adopt the tool. The low variability in most constructs supports the reliability of these averages. However, slightly higher deviations were observed for PU and BI, likely reflecting differences in participants' familiarity with the evaluated DSLs and the relative complexity of the tasks. This variation is consistent with expectations, as users with more DSL experience leaned to perceive higher usefulness and stronger intention to adopt the tool.

Breaking down PU, [Table 5](#) shows the subdimension scores. All scores were high, with a slight trend showing that general usefulness (C4 and C5) was rated marginally higher than task-specific measures. Task accuracy (B1, B2, and B3) and task efficiency (B4, B5, and B6) were similarly rated, with a slightly lower score due to a performance



**Table 4**  
Average survey results per TAM construct.

| TAM Construct                | Mean (1–5) | SD   |
|------------------------------|------------|------|
| Perceived Usefulness (PU)    | 4.38       | 0.77 |
| Perceived Ease of Use (PEOU) | 4.80       | 0.56 |
| Attitude Towards Use (ATU)   | 4.90       | 0.32 |
| Behavioral Intention (BI)    | 4.60       | 0.70 |

**Table 5**  
Subdimension scores for Perceived Usefulness (PU).

| PU Subdimension                         | Mean (1–5) | SD   |
|-----------------------------------------|------------|------|
| Task Accuracy (B1, B2, B3)              | 4.33       | 0.82 |
| Task Efficiency/Time Saved (B4, B5, B6) | 4.27       | 0.88 |
| General Usefulness (C4, C5)             | 4.60       | 0.52 |

issue with the GIS-related DSL for one participant, likely caused by the high specificity and greater complexity of this case. This breakdown helps explain the slightly higher standard deviation for PU in Table 4, as responses varied according to task complexity and participants' prior experience. Overall, participants perceived DSL-Xpert 2.0 as broadly useful and effective in facilitating DSL use across diverse domains and complexity levels.

In terms of qualitative feedback, all five participants agreed that the most valuable features of DSL-Xpert 2.0 were its ability to save time and ease entry into completely unfamiliar DSLs, even without prior knowledge of the domain. Regarding suggested improvements, four participants noted that the tool could be more flexible when interacting with the LLM, as instructions needed to be quite specific and attempts to perform more complex or generic tasks were sometimes less successful. Only one participant reported no issues or suggested improvements. These qualitative insights complement the quantitative findings: participants particularly appreciated the efficiency gains captured in the PU subdimension concerning time saved, while the slightly lower scores in task accuracy reflect the challenges encountered with more complex DSL tasks. Overall, both quantitative and qualitative results support the conclusion that the tool substantially facilitates the learning and use of DSLs, regardless of the previous user experience.

## 7. Conclusions and future work

This paper introduces DSL-Xpert 2.0, a tool designed to facilitate code generation for domain-specific languages (DSLs) with ease and speed, even for users without domain expertise. By using large language models (LLMs), DSL-Xpert 2.0 automates the conversion of natural language inputs into DSL code, enabling users to communicate directly with the model. This approach makes DSL adoption more accessible, even for those with limited technical knowledge.

DSL-Xpert 2.0 offers several customizable options to cater to a wide range of user needs. These include parameter configuration, model selection, integration modes (such as OpenAI, HuggingFace, WebLLM, and custom REST APIs), predefined model presets based on previous tests, and grammar definitions with validation of example usage to minimize errors in LLM-driven code generation. By supporting various models and offering flexible integration options, DSL-Xpert 2.0 serves as a transparent and adaptable solution for DSL code generation tasks.

To evaluate its effectiveness, we conducted an extensive assessment of various LLMs and hyperparameters, gaining valuable insights into their interaction with DSL-Xpert 2.0. The results demonstrate that while

the tool can achieve promising performance, there is potential for further optimization. Refinement of model selection and configuration remains a key area for future enhancement.

Looking ahead, the tool's adaptability to a wider variety of LLMs and grammar specifications stands as an important avenue for improvement. Although users can add any DSL to the tool, automatic validation is currently limited to Langium vocabularies. Expanding the number of supported validation frameworks would significantly increase the tool's versatility. In addition, experimenting with prompt structures, grammatical variations, and few-shot learning examples could further enhance the robustness of DSL-Xpert 2.0, helping to identify optimal configurations for diverse DSL use cases.

A key future goal for DSL-Xpert 2.0 is to evaluate it in real-world settings, especially in production systems, where user input tends to be more complex and inference speed is critical. To evaluate the accuracy, latency, and cost of the tool model in real-world DSL applications, future research should focus on end-to-end pipelines. These pipelines would offer a thorough understanding of how the tool performs in real-world conditions. One current limitation of this work is the lack of a direct comparison with general-purpose LLMs used in isolation, such as ChatGPT. Addressing this gap through a dedicated user-oriented study, comparing DSL-Xpert 2.0 with off-the-shelf models in practical DSL tasks, would help clarify the specific benefits of features like grammar validation and model flexibility. Moreover, a detailed error analysis of failure scenarios, particularly with complex DSL structures, will reveal ongoing challenges and help direct targeted improvements.

It is important to note that both the evaluation and the user survey conducted in this study were limited by the small number of DSLs considered. These DSLs were selected to reflect a range of structural and syntactic designs, including imperative, declarative, logic-based, and hierarchical formats. However, we acknowledge that this does not exhaust the whole design space of possible DSLs, although we believe it constitutes a sufficiently varied and representative sample to validate the tool's applicability to a wide range of DSL types. In any case, expanding this scope in future evaluations will provide a clearer picture of the tool's potential and performance across a broader set of DSLs.

Beyond the tool itself, several research challenges could contribute to the field of DSL code generation. For instance, while quantized models have shown promise in balancing efficiency and performance, further work on model compression and pruning could make smaller models more competitive for DSL tasks by reducing computational costs and latency. In addition, investigating automated hyperparameter tuning and model alignment techniques will be valuable for fine-tuning the tool's performance and ensuring its adaptability to a variety of DSLs.

## CRedit authorship contribution statement

**Victor Lamas:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Methodology, Investigation, Data curation, Conceptualization. **Daniel Garcia-Gonzalez:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Conceptualization. **Luca Sala:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Miguel R. Luaces:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition, Conceptualization.

## Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT in order to improve the readability and language of the manuscript. After using this service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.



## Funding

Funding for open access charge: Universidade da Coruña/CISUG. CITIC, as a center accredited for excellence within the Galician University System and a member of the CIGUS Network, has been partially funded by GAIN/Xunta de Galicia: [ED431G 2023/01]; the authors have been partially funded by MCIN/AEI/10.13039/501100011033 and “NextGenerationEU”/PRTR: [PLAGEMIS: TED2021-129245B-C21]; and MCIN/AEI/ 10.13039/501100011033 and EU/ERDF A way of making Europe: [EarthDL: PID2022-141027NB-C21]. GRC: ED431C 2025/34, partially funded by GAIN/Xunta de Galicia.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Appendix. Short user survey document

Please answer the following questions based on your experience using DSL-Xpert 2.0 with the DSLs you tested.

### Section A. DSL background

For questions A1–A3, use a 1–5 scale, where: 1 = Beginner 2 = Basic 3 = Intermediate 4 = Advanced 5 = Expert

- A1. My experience level with DSL-1 is:
- A2. My experience level with DSL-2 is:
- A3. My experience level with DSL-3 is:

### Section B. Specific tool performance

For questions B1–B6, use a 1–5 scale, where: 1 = Strongly disagree 2 = Disagree 3 = Neutral 4 = Agree 5 = Strongly agree

- B1. The generated code for DSL-1 matched what I intended.
- B2. The generated code for DSL-2 matched what I intended.
- B3. The generated code for DSL-3 matched what I intended.
- B4. Using DSL-Xpert 2.0 saved me time compared to writing DSL-1 code manually.
- B5. Using DSL-Xpert 2.0 saved me time compared to writing DSL-2 code manually.
- B6. Using DSL-Xpert 2.0 saved me time compared to writing DSL-3 code manually.

### Section C. General tool evaluation

For questions C1–C9, use a 1–5 scale, where: 1 = Strongly disagree 2 = Disagree 3 = Neutral 4 = Agree 5 = Strongly agree

- C1. It was easy to get started with DSL-Xpert 2.0.
- C2. The interface (prompts, validation, corrections) was intuitive.
- C3. The grammar validation and corrections were easy to understand and use.
- C4. DSL-Xpert 2.0 increases my productivity when working with DSLs.
- C5. DSL-Xpert 2.0 produces results that are of acceptable quality for my tasks.
- C6. Using DSL-Xpert 2.0 is a pleasant experience.
- C7. I would use DSL-Xpert 2.0 in my regular DSL work.
- C8. I would recommend DSL-Xpert 2.0 to others working with DSLs.
- C9. Overall satisfaction with DSL-Xpert 2.0.

## Section D. Open feedback

For questions D1–D2, just write a short text:

- D1. What did you find most useful about DSL-Xpert 2.0?
- D2. What would you improve in future versions?

## Data and materials availability

The tool is available to download at <https://github.com/lbdudc/dsl-xpert>, along with the user manual. Regarding the text-to-DSL experiments, the framework used is available at <https://github.com/lucasala1997/text-to-DSL>.

## References

- [1] M. Mernik, J. Heering, A. Sloane, When and how to develop domain-specific languages, *ACM Comput. Surv. (CSUR)* 37 (2005) 316–344.
- [2] M. Fowler, *Domain-Specific Languages*, Pearson Education, 2010.
- [3] T. Kosar, N. Oliveira, M. Mernik, V. Pereira, M. Črepinšek, C. Da, R. Henriques, Comparing general-purpose and domain-specific languages: An empirical study, *Comput. Sci. Inf. Syst.* 7 (2010) 247–264.
- [4] A. Iung, J. Carbonell, L. Marchezan, E. Rodrigues, M. Bernardino, F. Basso, B. Medeiros, Systematic mapping study on domain-specific language development tools, *Empir. Softw. Eng.* 25 (2020) 4205–4249.
- [5] A. Bragança, I. Azevedo, N. Bettencourt, C. Morais, D. Teixeira, D. Caetano, Towards supporting SPL engineering in low-code platforms using a DSL approach, in: *Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, 2021, pp. 16–28.
- [6] G. Karsai, H. Krahn, C. Pinkernell, B. Rumpe, M. Schindler, S. Völkel, Design guidelines for domain specific languages, 2014, arXiv preprint [arXiv:1409.2378](https://arxiv.org/abs/1409.2378).
- [7] A. Salman, M. Al-Jawad, W. Al. Tameemi, Domain-specific languages for IoT: Challenges and opportunities, *IOP Conf. Ser.: Mater. Sci. Eng.* 1067 (2021) 012133.
- [8] M. Villanueva, F. Valverde, O. Pastor, Involving end-users in the design of a domain-specific language for the genetic domain, in: *Information System Development: Improving Enterprise Communication*, 2014, pp. 99–110.
- [9] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, 2023, arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774).
- [10] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al., The llama 3 herd of models, 2024, arXiv preprint [arXiv:2407.21783](https://arxiv.org/abs/2407.21783).
- [11] T. Brown, Language models are few-shot learners, 2020, arXiv preprint [arXiv:2005.14165](https://arxiv.org/abs/2005.14165).
- [12] X. Gu, M. Chen, Y. Lin, Y. Hu, H. Zhang, C. Wan, Z. Wei, Y. Xu, J. Wang, On the effectiveness of large language models in domain-specific code generation, *ACM Trans. Softw. Eng. Methodol.* (2024).
- [13] V. Lamas, M. R. Luaces, D. Garcia-Gonzalez, DSL-xpert: LLM-driven generic DSL code generation, in: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pp. 16–20.
- [14] Openai platform, 2024, Available at: <https://platform.openai.com/docs/overview>. (Accessed 25 November 2024).
- [15] Hugging face website, 2024, Available at: <https://huggingface.co/>. (Accessed 25 November 2024).
- [16] Webllm website, 2024, Available at: <https://webllm.mlc.ai/>. (Accessed 25 November 2024).
- [17] D. Busch, G. Nolte, A. Balczyk, B. Steffen, ChatGPT in the loop: a natural language extension for domain-specific modeling languages, in: *International Conference on Bridging the Gap Between AI and Reality*, 2023, pp. 375–390.
- [18] R. Jain, W. Ni, J. Sunshine, Generating domain-specific programs for diagram authoring with large language models, in: *Companion Proceedings of the 2023 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, 2023, pp. 70–71.
- [19] J. Wang, Guiding large language models to generate computer-parsable content, 2024.
- [20] A. Kamath, R. Das, A survey on semantic parsing, 2018, arXiv preprint [arXiv:1812.00978](https://arxiv.org/abs/1812.00978).
- [21] Meta llama models, 2024, Available at: <https://github.com/meta-llama>. (Accessed 06 May 2024).
- [22] B. Wang, Z. Wang, X. Wang, Y. Cao, R.A. Saurous, Y. Kim, Grammar prompting for domain-specific language generation with large language models, in: *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [23] D. McCracken, E. Reilly, Backus-naur form (bnf), *Encycl. Comput. Sci.* (2003) 129–131.

- [24] K. Okuda, S. Amarasinghe, Askit: Unified programming interface for programming with large language models, in: 2024 IEEE/ACM International Symposium on Code Generation and Optimization, CGO, 2024, pp. 41–54.
- [25] L. Beurer-Kellner, M. Fischer, M. Vechev, Guiding LLMs the right way: Fast, non-invasive constrained generation, 2024, arXiv preprint [arXiv:2403.06988](https://arxiv.org/abs/2403.06988).
- [26] I. Zahid, S. Joudar, Enhancing XML-based compiler construction with large language models: A novel approach, *Mesopotamian J. Big Data* 2024 (2024) 23–39.
- [27] A. Parnami, M. Lee, Learning from few examples: A summary of approaches to few-shot learning, 2022, arXiv preprint [arXiv:2203.04291](https://arxiv.org/abs/2203.04291).
- [28] N. Baumann, J. Diaz, J. Michael, L. Netz, H. Nqiri, J. Reimer, B. Rumpe, Combining retrieval-augmented generation and few-shot learning for model synthesis of uncommon DSLs, in: *Modellierung 2024 Satellite Events*, pp. 10–18420.
- [29] J. Howard, S. Ruder, Universal language model fine-tuning for text classification, 2018, arXiv preprint [arXiv:1801.06146](https://arxiv.org/abs/1801.06146).
- [30] K. VM, H. Warrior, Y. Gupta, et al., Fine tuning LLM for enterprise: Practical guidelines and recommendations, 2024, arXiv preprint [arXiv:2404.10779](https://arxiv.org/abs/2404.10779).
- [31] H. Li, Y. Su, D. Cai, Y. Wang, L. Liu, A survey on retrieval-augmented text generation, 2022, arXiv preprint [arXiv:2202.01110](https://arxiv.org/abs/2202.01110).
- [32] N. Bassamzadeh, C. Methani, A comparative study of DSL code generation: Fine-tuning vs. Optimized retrieval augmentation, 2024, arXiv preprint [arXiv:2407.02742](https://arxiv.org/abs/2407.02742).
- [33] N. Bassamzadeh, C. Methani, Plan with code: Comparing approaches for robust NL to DSL generation, 2024, arXiv preprint [arXiv:2408.08335](https://arxiv.org/abs/2408.08335).
- [34] B. Darnell, H. Chopra, A. Councilman, D. Grove, Y. Wang, V. Adve, An empirical comparison of code generation approaches for ansible, in: *Proceedings of the ACM/IEEE 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering*, 2024, pp. 1–6.
- [35] P. Kogler, A. Falkner, S. Sperl, Reliable generation of formal specifications using large language models, in: *SE 2024-Companion*, 2024, pp. 141–153.
- [36] M. M. Mosthaf, A. Wasowski, From a natural to a formal language with DSL assistant, in: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 541–549.
- [37] Langium, 2025, Available at: <https://langium.org>. (Accessed 03 February 2025).
- [38] R. Clari , J. Cabot, Model-driven prompt engineering, in: *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems, MODELS*, 2023, pp. 47–54.
- [39] C. Braghin, M. Lilli, E. Riccobene, M. Baba, Kant: A domain-specific language for modeling security protocols, in: *MODELSWARD*, pp. 62–73.
- [40] D. Merkel, Docker: lightweight linux containers for consistent development and deployment, *Linux J.* 2014 (2014) 2.
- [41] Docker compose, 2025, Available at: <https://docs.docker.com/compose/>. (Accessed 03 February 2025).
- [42] Vue.js, 2025, Available at: <https://vuejs.org>. (Accessed 03 February 2025).
- [43] Node.js, 2025, Available at: <https://nodejs.org>. (Accessed 03 February 2025).
- [44] MongoDB, 2025, Available at: <https://www.mongodb.com>. (Accessed 03 February 2025).
- [45] Mongoose, 2025, Available at: <https://mongoosejs.com>. (Accessed 03 February 2025).
- [46] T. Wolf, Transformers: State-of-the-art natural language processing, 2020, arXiv preprint [arXiv:1910.03771](https://arxiv.org/abs/1910.03771).
- [47] V. Lamas, A. Corti as, M.R. Luaces, Low-code framework for IoT data warehousing and visualization, *Comput. Geosci.* 205 (2025) 105998.
- [48] M. Evtikhiev, E. Bogomolov, Y. Sokolov, T. Bryksin, Out of the bleu: how should we assess quality of the code generation models? *J. Syst. Softw.* 203 (2023) 111741.
- [49] OpenAI, Hello, GPT-4O!, 2025, Available at: <https://openai.com/index/hello-gpt-4o/>. (Accessed 14 January 2025).
- [50] OpenAI, GPT-4O mini: Advancing cost-efficient intelligence, 2025, Available at: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>. (Accessed 14 January 2025).
- [51] A. Lozhkov, R. Li, L. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, et al., Starcoder 2 and the stack v2: The next generation, 2024, arXiv preprint [arXiv:2402.19173](https://arxiv.org/abs/2402.19173).
- [52] Qwen Team, QWQ-32B preview, 2024, Available at: <https://qwenlm.github.io/blog/qwq-32b-preview/>. (Accessed 14 January 2024).
- [53] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, et al., Qwen2, 5 Technical Report, 2024, arXiv preprint [arXiv:2412.15115](https://arxiv.org/abs/2412.15115).
- [54] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, et al., Code llama: Open foundation models for code, 2023, arXiv preprint [arXiv:2308.12950](https://arxiv.org/abs/2308.12950).
- [55] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocar, M. Debbah,  . Goffinet, D. Hesslow, J. Launay, Q. Malartic, et al., The falcon series of open language models, 2023, arXiv preprint [arXiv:2311.16867](https://arxiv.org/abs/2311.16867).
- [56] A. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. Chaplot, D. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al., Mistral 7B, 2023, arXiv preprint [arXiv:2310.06825](https://arxiv.org/abs/2310.06825).
- [57] Mistral large 2407 announcement, 2024, Available at: <https://mistral.ai/en/news/mistral-large-2407>. (Accessed 25 November 2024).
- [58] Mistral A.I. Team, Mistral NeMo: our new best small model, 2025, Available at: <https://mistral.ai/news/mistral-nemo/>. (Accessed 14 January 2025).
- [59] NVIDIA, Llama-3.1-nemotron-70b-instruct, 2025, Available at: [https://build.nvidia.com/nvidia/llama-3\\_1-nemotron-70b-instruct](https://build.nvidia.com/nvidia/llama-3_1-nemotron-70b-instruct). (Accessed 14 January 2025).
- [60] S. Han, H. Mao, W. Dally, Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, 2015, arXiv preprint [arXiv:1510.00149](https://arxiv.org/abs/1510.00149).
- [61] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR*, 2018, pp. 2704–2713.
- [62] M. Nagel, M. Baalen, T. Blankevoort, M. Welling, Data-free quantization through weight equalization and bias correction, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision, ICCV*, 2019, pp. 1325–1334.
- [63] J. Wang, Y. Gong, X. Liu, M. Feng, L. Yuan, L. Wang, Q. Chen, T. Zhang, J. Yan, N. Sun, HAQ: Hardware-aware automated quantization with mixed precision, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR*, 2019, pp. 8612–8620.
- [64] M. Nagel, R. Amjad, L. Llad s, C. Louizos, T. Blankevoort, Up or down? Adaptive rounding for post-training quantization, 2020, arXiv preprint [arXiv:2004.10568](https://arxiv.org/abs/2004.10568).
- [65] Y. Lin, N. Kitaev, C. Manning, Dynamic quantization for NLP applications, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP*, 2020, pp. 4273–4278.
- [66] F. Davis, R. Bagozzi, P. Warshaw, Technology acceptance model, *J. Manag. Sci.* 35 (1989) 982–1003.
- [67] N. Maranguni , A. Grani , Technology acceptance model: a literature review from 1986 to 2013, *Univers. Access Soc.* 14 (2015) 81–95.
- [68] Langium playground, 2025, Available at: <https://langium.org/playground/>. (Accessed 24 October 2025).